

Milestone 3: Model & Infrastructure Optimization

Disaster Tweet Classifier, SYS-304 Scalable Algorithms and Infrastructure

23 September 2026

Repository: <https://github.com/Dojbat/COM-104>

1. Summary

Under 40 concurrent requests, the optimized Phase 3 system served **6.0x the throughput** of the naive Phase 2 system (49.0 vs 8.1 req/s) and cut mean latency from 4.7 s to 0.56 s. For repeated tweets served from the Redis cache, throughput rose to **10.2x** (82.4 req/s). The optimized stack, including Redis, used **2.5–3.2x less memory** under load, at a cost of 3.5 points of accuracy.

Metric	Phase 2 (naive)	Phase 3 (optimized)	Change
Throughput, 40 concurrent, unique texts	8.1 req/s	49.0 req/s	6.0x
Throughput, 40 concurrent, repeated texts	8.1 req/s (no cache)	82.4 req/s	10.2x
Mean latency, 40 concurrent, unique texts	4,665 ms	555 ms	8.4x lower
p95 latency, 40 concurrent, unique texts	4,810 ms	717 ms	6.7x lower
Single-request model latency (mean)	252.8 ms	43.8 ms	5.8x lower
Container memory under load	848–1,086 MiB	341 MiB (backend 327 + Redis 14)	2.5–3.2x smaller
Accuracy on 200 <code>train.csv</code> tweets	88.5%	85.0%	–3.5 pts

Concurrent figures are the median of 3 runs. All numbers were measured on one laptop on 23 September 2026 (setup in section 5).

2. What changed from Phase 2

Model-level (Week 5), in `milestone_3/export_onnx.py`:

- **Merged the LoRA adapter** into the BERTweet base weights (`merge_and_unload`), removing the PEFT wrapper and adapter matrix multiplications at inference time.
- **Exported to ONNX** with `optimum` and served it with ONNX Runtime instead of eager PyTorch.
- **Quantized weights from FP32 to INT8** with `quantize_dynamic(per_channel=True)`. Per-tensor quantization made predictions collapse to near-constant output; per-channel fixed it.

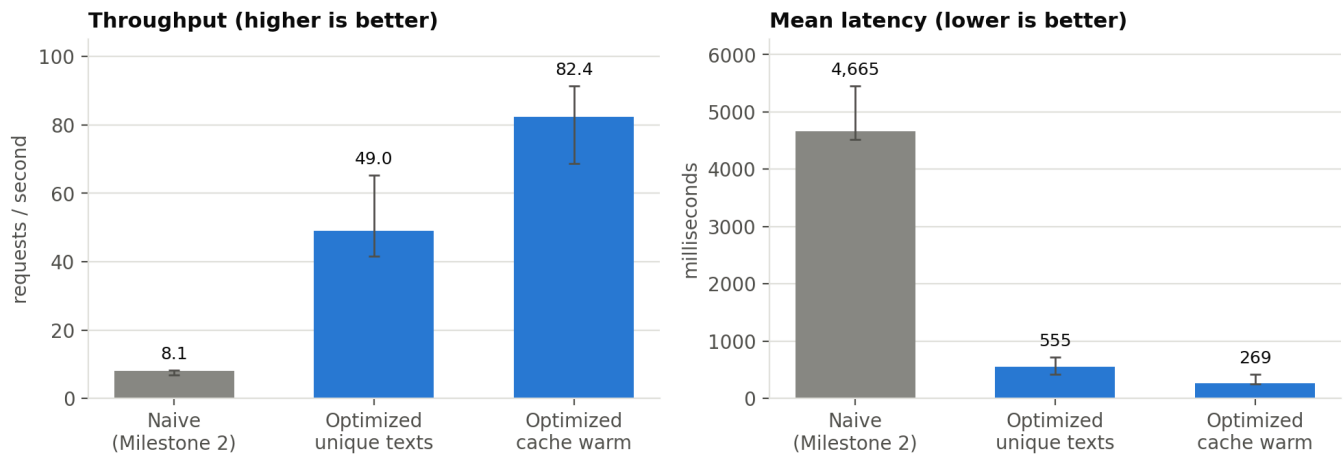
System-level (Week 6), in `milestone_3/backend/app/`:

- **Dynamic batching** (`batcher.py`): concurrent requests wait up to 20 ms, or until 16 have queued, and then run as one `predict_batch()` call instead of one forward pass each.
- **Redis exact-match cache** (`cache.py`): the key is the SHA-256 of the tweet text, with a 1-hour TTL. A repeated tweet skips the model entirely. If Redis is down, requests still get a model answer.
- **Two-stage Docker build**: the builder stage creates the INT8 model, and the runtime image ships only that file, ONNX Runtime and the tokenizer (no PyTorch). `docker-compose.yml` adds a `redis:7-alpine` service with a health check.

3. Benchmark results

3.1 Throughput and latency under concurrent load

40 concurrent POST /predict requests — median of 3 runs, whiskers = min-max



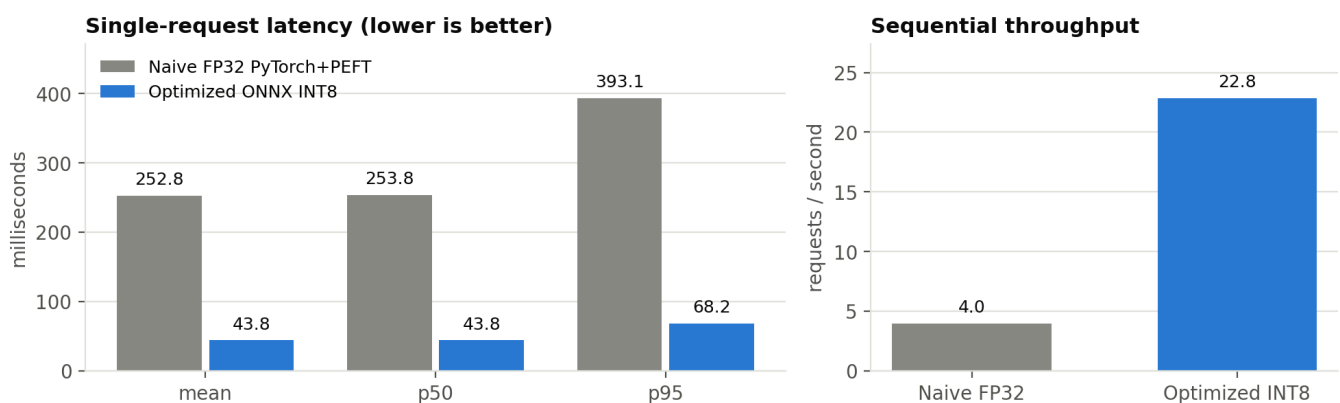
Throughput and mean latency for 40 concurrent requests

The naive backend handles 40 concurrent requests no faster than one at a time: 8.1 req/s, and every request waits about 4.7 s. FastAPI accepts all 40 connections, but each one runs its own model call on the CPU, so they queue behind each other. Under load, the naive container ran 169 processes and threads, compared with 18 for the optimized backend.

Batching turns those 40 separate forward passes into a few batched calls. Combined with the faster INT8 model, that gives 6.0x the throughput. With the cache warm, all 40 requests were cache hits and no model call ran. A single cache hit takes 2.2 ms inside the container, so most of the remaining ~270 ms is the load client and the HTTP handling of 40 simultaneous connections.

3.2 Single-request latency (model only)

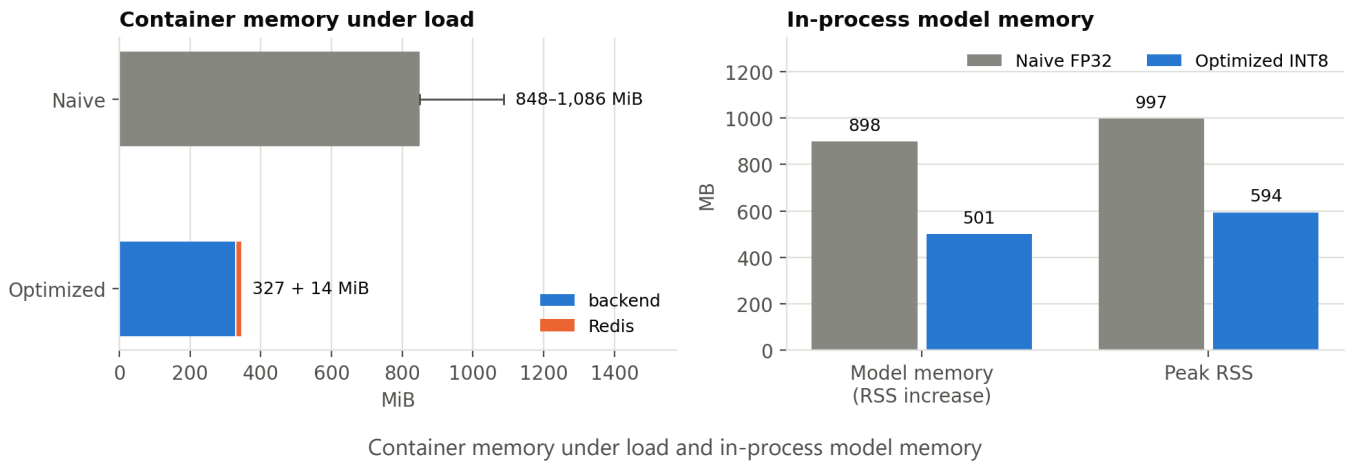
In-process benchmark, 200 tweets sent one at a time (benchmark.py)



Single-request latency and sequential throughput, naive vs optimized model

This test sends 200 tweets one at a time straight to the model, with no HTTP, so it isolates the Week 5 changes. The INT8 ONNX model is 5.8x faster at the mean, p50 and p95 alike. Model load time fell from 4.90 s to 2.34 s.

3.3 Memory footprint

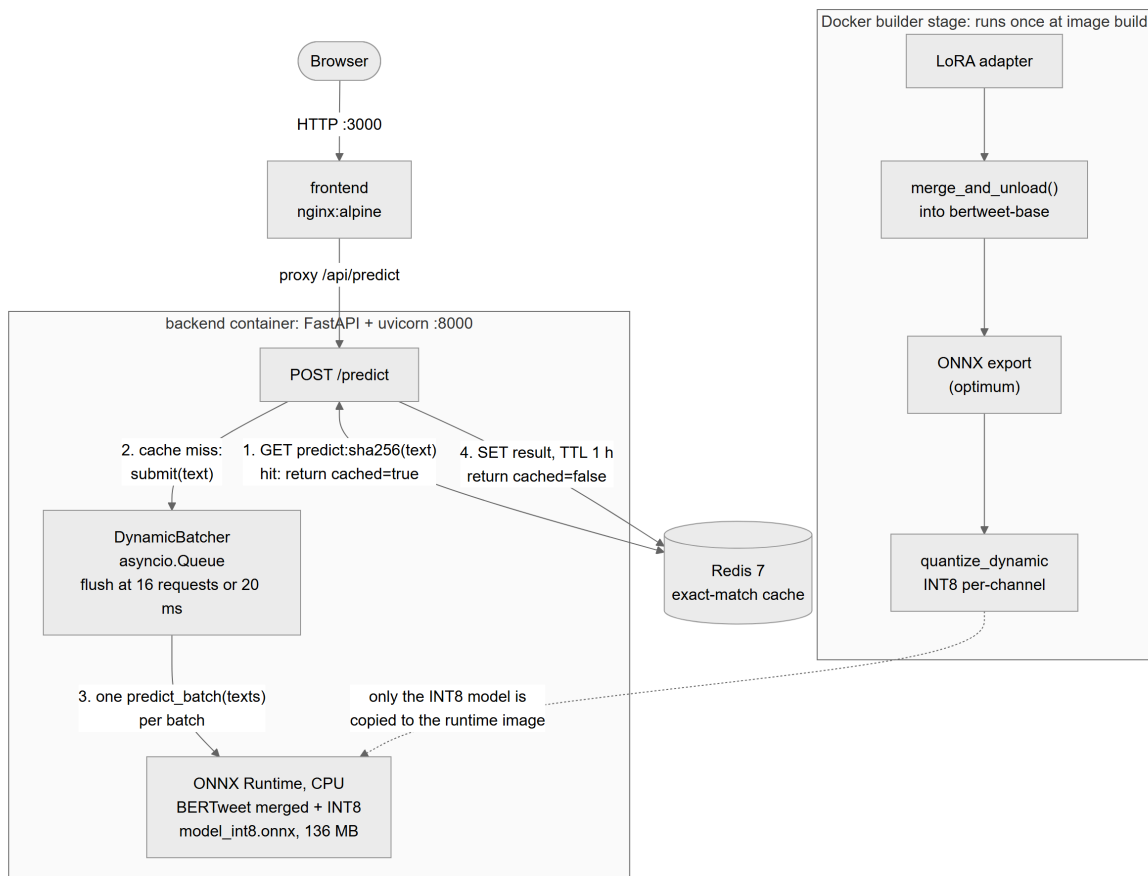


The optimized backend plus Redis uses 341 MiB under load, against 848–1,086 MiB for the naive backend. In-process, the model’s memory drops from 898 MB to 501 MB (1.8x). On disk, the served model is one self-contained 136 MB file, compared with a 3.6 MB adapter plus a ~540 MB base checkpoint (about 4x smaller).

3.4 Accuracy trade-off

On the same 200 tweets, accuracy fell from 88.5% to 85.0%, and the two models agreed on 91.5% of predictions. These tweets come from `train.csv`, which was used for fine-tuning, so the agreement rate is the fairer measure of how much quantization changed the model. We consider a 3.5-point drop acceptable for a 5.8x faster, 4x smaller model; a held-out test set would be needed to confirm it.

4. Architecture of the optimized system



Architecture diagram of the optimized system

The browser talks only to nginx, which proxies `/api/predict` to the FastAPI backend. For each request, the backend:

1. Looks the tweet up in Redis and returns immediately on a hit (`cached: true`).
2. On a miss, puts the request on the dynamic batcher's queue.
3. The batcher sends up to 16 queued tweets to ONNX Runtime in one `predict_batch()` call.
4. Stores the result in Redis for 1 hour and returns it (`cached: false`).

The INT8 model is built once in the Docker builder stage (merge, export, quantize), and only that file is copied into the runtime image. The diagram source is `milestone_3/report/architecture.mmd` (Mermaid.js).

5. Test setup and limitations

- **Machine:** Intel Core i7-1185G7 (4 cores, 8 threads), 16 GB RAM, Windows 11, Docker Desktop (WSL2, 8 CPUs, 7.6 GiB).
- **Concurrent load:** `milestone_3/backend/loadtest.py`, 40 concurrent `POST /predict`, 3 runs per scenario. The load client ran in a container on the compose network, because Docker Desktop's port forwarding added about 48 ms per request when running from Windows. Redis was flushed before each unique-text run, and only one backend was under load at a time.
- **Memory:** sampled with `docker stats` during a load run.
- **Model benchmark:** `python milestone_3/benchmark.py --num-samples 200`; each model ran in its own process.
- **Raw results:** `milestone_3/results/loadtest_results.json` and `milestone_3/results/benchmark_summary_2026-09-23.json`. Regenerate the charts with `python milestone_3/report/make_charts.py`.
- **Limitation:** the laptop was on battery during these runs and its CPU was clocked down to about 1 GHz. Absolute latency and throughput are therefore 2–3x worse than an earlier run recorded in the repository README (131.7 req/s optimized vs 12.3 naive, a 10.7x gain). The memory and accuracy figures match that run exactly.